ForkSysInfo TCP Client Server Lab Report

Repository: https://github.com/mimimaomao1117/forksysinfo

Date: November 13 2026

1 Test environment and features

This project implements a TCP system information service in C. The server uses `fork()` to create one child for each connection. No threads are used. The client and server both dynamically link `libnetsys.so`.

The program was tested in Linux under QEMU on the Windows computer used for this assignment. This command-line run uses Linux 6.12.117-0-virt, GCC 14.2.0, GNU Make 4.4.1, Python 3.12.16, tcpdump 4.99.5 and TShark 4.4.7. The screenshots are direct captures of the Linux virtual console, and their output is saved in `evidence-cli/`. Earlier records remain separately in `evidence/` and `evidence-recheck/`.

SYSINFO returns the Linux hostname, kernel, architecture, worker PID, uptime and memory information. PING checks the connection, ECHO returns text, SLEEP simulates work, and QUIT closes a session. SYSINFO reports what the Linux guest sees.

2 Server design and message format

The parent creates the listening socket and calls `accept()`. After each accepted connection, it calls `fork()`. The child handles that connection while the parent immediately returns to accepting more clients. The parent closes its copy of the connected socket, and the child closes its copy of the listening socket. The parent collects completed children with `waitpid(..., WNOHANG)`.

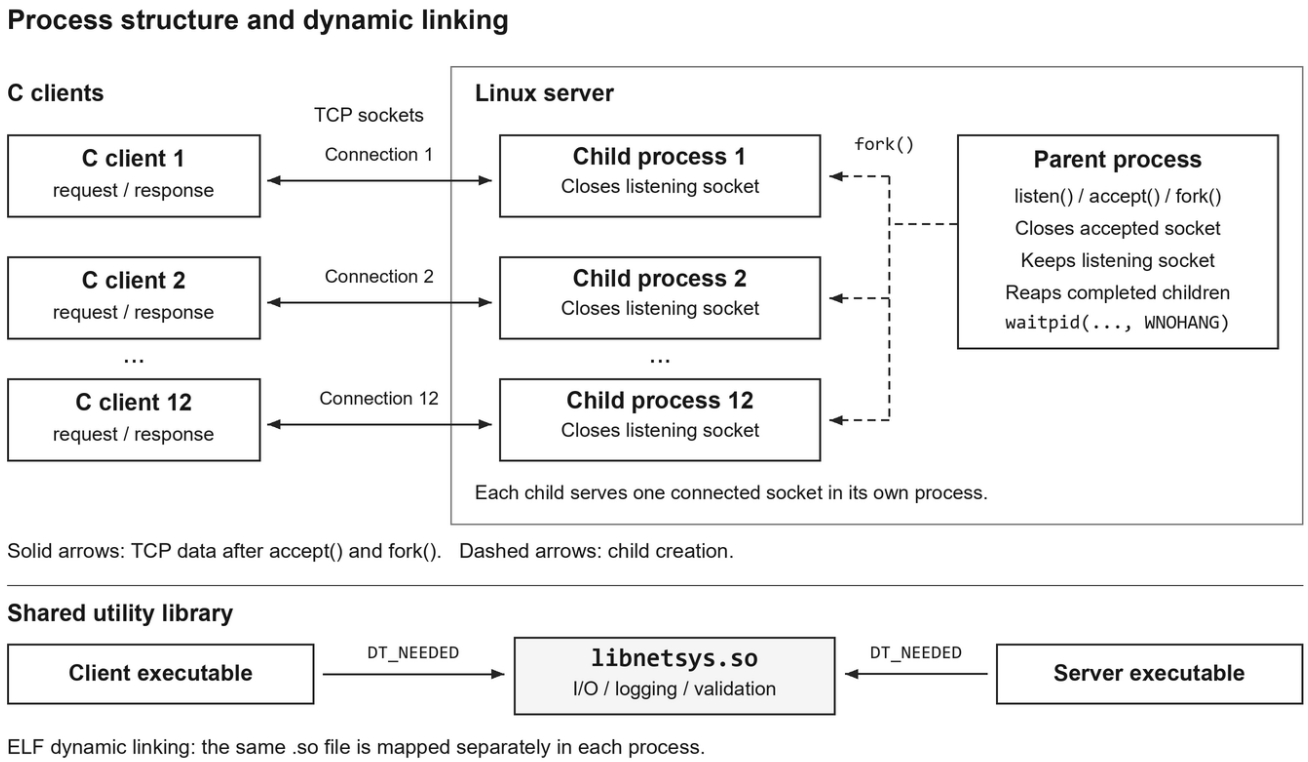


Figure 1. Parent process and child processes for separate client connections

A child can process several requests on one TCP connection. It exits after QUIT, EOF or a frame error. The command-line client sends one request, prints one reply and then closes its connection.

TCP is a byte stream. One send does not necessarily correspond to one receive. Each application message therefore begins with a 4-byte unsigned length in network byte order, followed by 1 to 4096 payload bytes. The terminating C NUL is not sent. A PING request is `00 00 00 04 50 49 4e 47`.

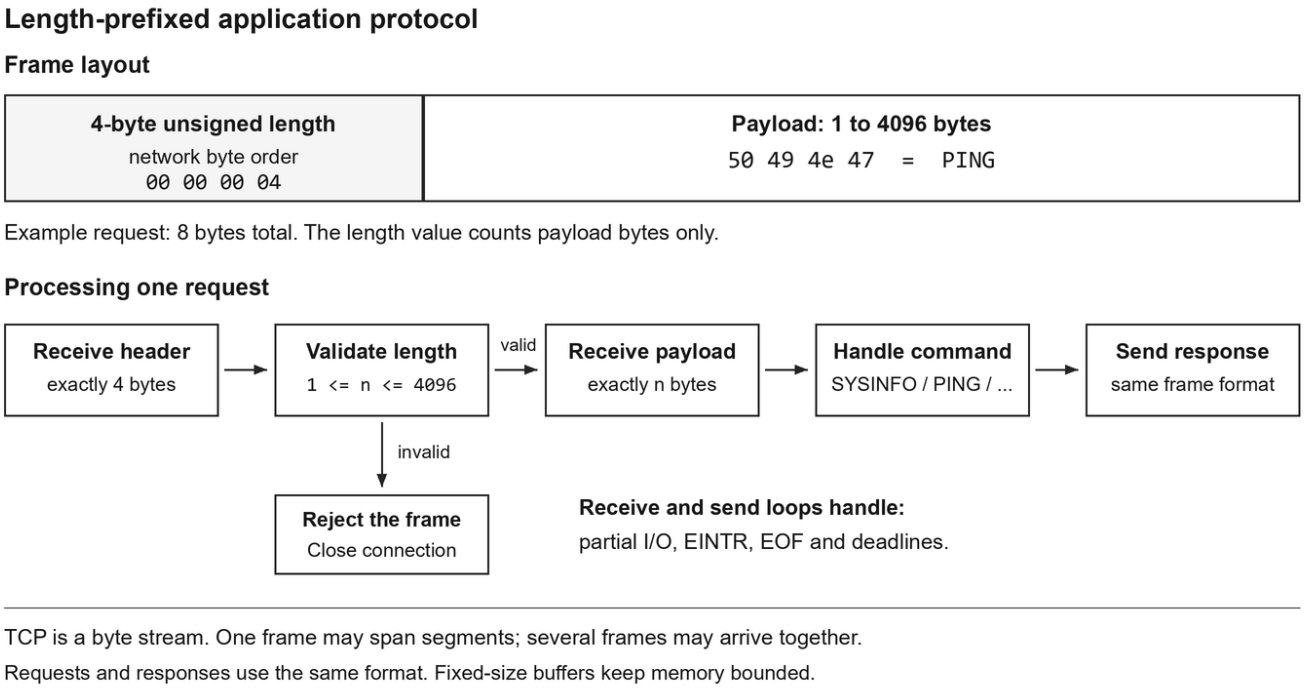


Figure 2. Application frame fields and the complete PING request

The main source files are:

- `src/server.c`: `main()` accepts and forks; `serve_client()` runs the session; `handle_command()` selects the operation; `system_info()` calls uname, sysinfo and gethostname.

- `src/client.c`: parses the options, connects, sends one request and displays the response.

- `src/netsys.c`: implements shared framing, socket setup, deadlines, numeric validation and logging. `include/netsys.h` declares the interface and debug macro.

- `Makefile`: builds the shared library and both executables in separate debug and release directories.

3 Build and system information request

The Makefile enables `-Wall -Wextra -Wpedantic -Werror`, so a compiler warning fails the build.

```
make DEBUG=1
make DEBUG=0
```

```
UTC 2026-11-13T05:30:58+00:00
# python3 scripts/cli_evidence.py build
# uname -srmo
Linux 6.12.117-0-virt x86_64 Linux
# make -B DEBUG=1 all
mkdir -p build/debug
cc -Iinclude -DENABLE_DEBUG=1 -std=c11 -Wall -Wextra -Wpedantic -Werror -O0 -g3 -fPIC -c src/netsys.c -o b
uild/debug/netsys.o
cc -shared  -Wl,-soname,libnetsys.so build/debug/netsys.o -o build/debug/libnetsys.so
cc -Iinclude -DENABLE_DEBUG=1 -std=c11 -Wall -Wextra -Wpedantic -Werror -O0 -g3 src/server.c -Lbuild/debug
 -lnetsys -Wl,-rpath,'$ORIGIN'  -o build/debug/server
cc -Iinclude -DENABLE_DEBUG=1 -std=c11 -Wall -Wextra -Wpedantic -Werror -O0 -g3 src/client.c -Lbuild/debug
 -lnetsys -Wl,-rpath,'$ORIGIN'  -o build/debug/client
# make -B DEBUG=0 all
mkdir -p build/release
cc -Iinclude -DENABLE_DEBUG=0 -std=c11 -Wall -Wextra -Wpedantic -Werror -O2 -fPIC -c src/netsys.c -o build
/release/netsys.o
cc -shared  -Wl,-soname,libnetsys.so build/release/netsys.o -o build/release/libnetsys.so
cc -Iinclude -DENABLE_DEBUG=0 -std=c11 -Wall -Wextra -Wpedantic -Werror -O2 src/server.c -Lbuild/release -
lnetsys -Wl,-rpath,'$ORIGIN'  -o build/release/server
cc -Iinclude -DENABLE_DEBUG=0 -std=c11 -Wall -Wextra -Wpedantic -Werror -O2 src/client.c -Lbuild/release -
lnetsys -Wl,-rpath,'$ORIGIN'  -o build/release/client
_
```

Figure 3. Building debug and release executables in the Linux command line

Start the server in one terminal and run the client in another:

```
./build/debug/server --port 9090 --debug
./build/debug/client --port 9090 SYSINFO
./build/debug/client --port 9090 ECHO "Hello from C"
```

The PID in a SYSINFO response belongs to the child handling that connection. A new client connection normally receives a different worker PID. SLEEP accepts an integer from 0 to 10000 milliseconds.

```
UTC 2026-11-13T05:31:57+00:00
# python3 scripts/cli_evidence.py sysinfo
server: build/debug/server --host 127.0.0.1 --port 19200 --timeout-ms 2000 --max
-clients 64
server parent PID=2123
# ./build/debug/client --port 19200 SYSINFO
OK SYSINFO
hostname=localhost
sysname=Linux
release=6.12.117-0-virt
machine=x86_64
pid=2125
uptime_seconds=334
ram_total_bytes=1546407936
ram_free_bytes=1320095744
# ./build/debug/client --port 19200 PING
OK PONG pid=2127
# ./build/debug/client --port 19200 ECHO 'hello from the command line'
OK hello from the command line
```

Figure 4. Actual client requests and system information responses

4 Shared library and debug controls

The shared code is compiled with `-fPIC` and linked as `libnetsys.so`. Both executables use `-lnetsys`. Their RUNPATH contains `$ORIGIN`, so they find the library beside the executable without changing the system library path.

```
ldd build/debug/client
ldd build/debug/server
readelf -d build/debug/server
```

The compile-time `ENABLE_DEBUG` macro determines whether debug calls exist in the binary. The runtime `--debug` option controls whether those calls print anything. Startup, lifecycle and error messages remain available.

Build setting	Runtime option	Debug output
DEBUG=1	No --debug	No
DEBUG=1	With --debug	Yes
DEBUG=0	No --debug	No
DEBUG=0	With --debug	No

The command-line PING test produced 0, 8, 0 and 0 debug lines for these four combinations. Eight is the combined client and server count for that transaction, rather than a fixed count for every command. Separate `build/debug/` and `build/release/` directories prevent objects compiled with different settings from being mixed.

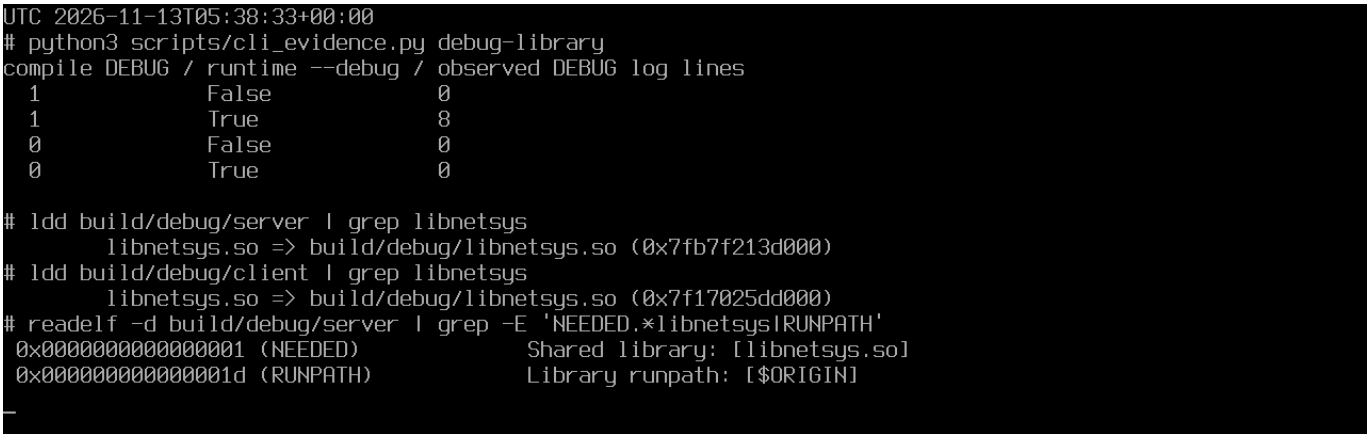


Figure 5. Debug combinations and dynamic library linkage checks

5 Twelve concurrent clients

The standalone command-line test launches 12 C clients together on port 19210. Each requests SLEEP 3000. While the requests overlap, ps and ss record the worker processes and established TCP connections. The test then checks every client's exit status and reply.

```
python3 scripts/cli_evidence.py concurrency
```

The snapshot shows 12 distinct workers under parent PID 2152. All clients received `OK SLEPT 3000` and exited with status 0. Total elapsed time was 3.175 seconds. Sequential waiting alone would take at least 12 times 3 seconds, or 36 seconds. The overlapping workers are the main concurrency proof; twelve successful lines alone would also be possible with sequential execution.

```
UTC 2026-11-13T05:33:15+00:00
# python3 scripts/cli_evidence.py concurrency
server parent PID=2152; client count=12; SLEEP=3000 ms
client command (12 subprocesses): ./build/debug/client --port 19210 SLEEP 3000
# ps -o pid,ppid,stat,comm --ppid 2152
  PID  PPID STAT COMMAND
 2155  2152 S    server
 2156  2152 S    server
 2159  2152 S    server
 2161  2152 S    server
 2163  2152 S    server
 2164  2152 S    server
 2167  2152 S    server
 2169  2152 S    server
 2170  2152 S    server
 2173  2152 S    server
 2174  2152 S    server
 2176  2152 S    server
client responses:
01  OK SLEPT 3000 pid=2155
02  OK SLEPT 3000 pid=2156
03  OK SLEPT 3000 pid=2159
04  OK SLEPT 3000 pid=2161
05  OK SLEPT 3000 pid=2163
06  OK SLEPT 3000 pid=2164
07  OK SLEPT 3000 pid=2167
08  OK SLEPT 3000 pid=2170
09  OK SLEPT 3000 pid=2169
10  OK SLEPT 3000 pid=2173
11  OK SLEPT 3000 pid=2174
12  OK SLEPT 3000 pid=2176
observed 12 workers at one time; elapsed=3.175 s
serial SLEEP lower bound: 12 x 3 = 36 s
```

Figure 6. Twelve worker processes and their client responses

On loopback, ss can display both endpoints of each connection. Twenty-four endpoints therefore do not mean twenty-four clients.

6 Abnormal conditions and paired tests

Normal mode handles partial I/O, EINTR, EOF, read and write deadlines, and SIGPIPE. It checks frame lengths and command arguments before using them. A frame deadline uses a monotonic clock and is not restarted whenever another byte arrives. The parent defaults to a limit of 64 workers and cleans up remaining children when it shuts down.

For comparison, `--unsafe` disables complete receive loops, receive deadlines and normal child reaping. Buffer limits remain enabled. Each paired test uses the same input, and the test harness cleans up both modes afterwards.

Test	Normal mode	--unsafe mode
Send the PING frame in 1, 2 and 5-byte fragments	Reads the full frame and replies	Short header causes the connection to fail
Connect without sending data; deadline 500 ms	Worker exits after the deadline	Worker keeps waiting
Complete PING and close the client	Parent reaps the worker	Worker remains in state Z

In the command-line run, fragmented PING returned `OK PONG pid=2189` normally; a later write in unsafe mode reported Broken pipe. After 1.2 seconds, the idle worker counts were 0 and 1. The unsafe worker remained in state S. After completing PING, the zombie counts were 0 and 1, with unsafe worker PID 2201 in state Z. The 1.2 seconds was the observation time, while the configured deadline was 500 ms.

```
UTC 2026-11-13T05:33:21+00:00
# python3 scripts/cli_evidence.py robustness
fragmented PING frame: send 1 byte, then 2 bytes, then 5 bytes
  robust: OK PONG pid=2189
  unsafe: BrokenPipeError: [Errno 32] Broken pipe

idle connection: send no bytes; timeout=500 ms; inspect after 1.2 s
  robust: active workers=0; children=[]
  unsafe: active workers=1; children=[(2195, 'S')]

completed PING client: inspect child state after 300 ms
  robust: zombies=0; children=[]
  unsafe: zombies=1; children=[(2201, 'Z')]
All temporary server processes have been stopped.
_
```

Figure 7. Command-line results for fragmentation timeout and zombie tests

Other checks cover length 0, 4097 and UINT32_MAX, embedded NUL, truncated payload, invalid commands and SLEEP arguments, peer reset, occupied port and refused connection. An admission test sets the worker limit to 2 and checks that the third connection receives SERVER_BUSY.

7 Packet capture and analysis

Tcpdump captures traffic on the Linux loopback interface. TShark reads the saved PCAP to show TCP flags, sequence and acknowledgment numbers, and payload bytes.

```
tcpdump -i lo -nn -s 0 -U \
  -w evidence-cli/loopback.pcap 'tcp port 19090'
tshark -r evidence-cli/loopback.pcap -Y 'tcp.stream == 0'
tshark -r evidence-cli/loopback.pcap \
  -Y 'tcp.len == 8 && tcp.dstport == 19090' -x
```

```
UTC 2026-11-13T05:38:37+00:00
# python3 scripts/cli_evidence.py tshark
# tshark --version | head -n1
TShark (Wireshark) 4.4.7 (Git commit e8a60fa2ff17).
# tshark -r evidence-cli/loopback.pcap -Y 'tcp.stream == 0' -T fields -E header=y -e frame.number -e tcp.s
rcport -e tcp.dstport -e tcp.flags -e tcp.seq -e tcp.ack -e tcp.len | awk '{printf "%-13s %-12s %-12s %-11
s %-9s %-9s %s\n", $1, $2, $3, $4, $5, $6, $7}'
frame.number  tcp.srcport  tcp.dstport  tcp.flags   tcp.seq   tcp.ack   tcp.len
1             55766        19090        0x0002      0         0         0
2             19090        55766        0x0012      0         1         0
3             55766        19090        0x0010      1         1         0
4             55766        19090        0x0018      1         1         11
5             19090        55766        0x0010      1         12        0
6             19090        55766        0x0018      1         12        167
7             55766        19090        0x0010      12        168       0
8             55766        19090        0x0011      12        168       0
9             19090        55766        0x0011      168       13        0
10            55766        19090        0x0010      13        169       0
```

Figure 8. TShark output showing handshake data transfer and connection close

The fresh `evidence-cli/loopback.pcap` is 3664 bytes and contains 4 connections and 40 packets. Eight packets carry application data, and no RST is present. Eight SYN flags include four SYN requests and four SYN ACK replies. Eight FIN flags represent the normal closing of the four connections. The requests are SYSINFO, PING, ECHO and SLEEP 50.

For TCP stream 0, frames 1 to 3 are the handshake. Frame 4 carries an 11-byte SYSINFO request: a 4-byte prefix and 7-byte command. Frame 6 carries 167 bytes: the prefix `00 00 00 a3` and a 163-byte system information response. Frames 8 to 10 close this connection with FIN ACK, FIN ACK and the final ACK.

PING begins with `00 00 00 04`, followed by `50 49 4e 47`. SYSINFO begins with `00 00 00 07`. TCP segments and application frames need not have matching boundaries. Reassembly must follow direction and

sequence numbers before the application length is decoded. ACK advancement covers both the 4-byte prefix and payload; SYN and FIN each consume one additional sequence number.

```
UTC 2026-11-13T05:35:37+00:00
# python3 scripts/cli_evidence.py tshark-hex
# tshark -r evidence-cli/loopback.pcap -Y 'tcp.len == 8 && tcp.dstport == 19090' -x
0000  00 00 00 00 00 00 00 00 00 00 00 00 08 00 45 00   ..............E.
0010  00 3c 33 e7 40 00 40 06 08 d3 7f 00 00 01 7f 00   .<3.@.@.........
0020  00 01 d9 e2 4a 92 64 95 51 f9 7f 27 a9 f2 80 18   ....J.d.Q..'....
0030  02 00 fe 30 00 00 01 01 08 0a c9 2c bf 29 48 9b   ...0.......,.)H.
0040  6d c4 00 00 00 04 50 49 4e 47                     m.....PING

# tshark -r evidence-cli/loopback.pcap -Y 'tcp.len == 8 && tcp.dstport == 19090' -T fields -e tcp.payload
0000000450494e47
```

Figure 9. Hexadecimal payload and application length prefixes

8 Reproduction and classroom demonstration

The public repository includes source, tests, reports, video, screenshots and reference packet records. Git excludes `build/`, `portable-linux/` and `runs/`. On Ubuntu or Debian:

```
sudo apt-get update
sudo apt-get install -y build-essential python3 tcpdump tshark iproute2 binutils procps git
git clone https://github.com/mimimaomao1117/forksysinfo.git
cd forksysinfo
sudo bash scripts/run_all.sh
```

The complete test needs capture permission, which is why the command uses sudo. Ordinary client and server requests can run as a normal user.

The full desktop bundle also includes offline Linux. `START_DEMO.cmd` updates the C source, rebuilds and runs the tests, then exports new evidence to `runs/<timestamp>/`. For a manual demonstration, open two `LINUX_SHELL.cmd` windows and run `sh scripts/demo.sh --server` in one and `sh scripts/demo.sh` in the other. Finish with `STOP_LINUX.cmd`. A plain GitHub clone does not include the runtime required by those Windows launchers.

The two-minute video is `media/ForkSysInfo_2min.mp4`. Complete C source and the Makefile are included in the report appendix and supplied as editable files. The demonstration uses loopback and does not implement TLS or user authentication.

9 Complete integration run

The complete suite passed all 11 test groups on Linux 6.12.117. It checks both builds, normal features, malformed input, concurrency, three robustness comparisons, admission control, debug output, shared-library linkage and actual loopback capture. The full result is `evidence-cli/results.json`.

```
UTC 2026-11-13T05:33:45+00:00
# python3 scripts/cli_evidence.py suite
PASS Linux environment and required tools
PASS Build debug and release
PASS Normal features and abnormal conditions
PASS Twelve simultaneous C clients and fork children
PASS Robustness comparison fragmented header
PASS Robustness comparison idle timeout
PASS Robustness comparison zombie reaping
PASS Bounded admission control
PASS Compile time and runtime debug matrix
PASS Dynamic shared library linkage
PASS Real loopback tcpdump capture and frame analysis
11/11 passed; evidence: /root/ForkSysInfo/evidence-cli
```

Figure 10. Complete integration run in the Linux console

The suite launches its own temporary processes. Its PIDs and timings are separate from the standalone SLEEP 3000 demonstration in Section 5. That demonstration and the other console examples are indexed in `evidence-cli/console_results.json`, with the command output in `evidence-cli/raw/`.

Full uncropped console PNG files are supplied in `media/cli/`. The PDF layout omits only unused terminal space below the output.

References

1. Linux man-pages fork(2): https://man7.org/linux/man-pages/man2/fork.2.html

2. Linux man-pages waitpid(2): https://man7.org/linux/man-pages/man2/waitpid.2.html

3. Linux man-pages tcp(7): https://man7.org/linux/man-pages/man7/tcp.7.html

4. Tcpdump manual: https://www.tcpdump.org/manpages/tcpdump.1.html

5. GNU Make manual: https://www.gnu.org/software/make/manual/make.html

Appendix Complete Source Code

The following listings contain the complete C source, shared header and Makefile. Line numbers refer to the submitted files. The test and demonstration scripts are included separately in the repository.

include/netsys.h

```
 1  #ifndef NETSYS_H
 2  #define NETSYS_H
 3
 4  #include <stddef.h>
 5  #include <stdint.h>
 6
 7  #ifndef ENABLE_DEBUG
 8  #define ENABLE_DEBUG 0
 9  #endif
10
11  #define NET_MAX_FRAME 4096U
12  #define NET_DEFAULT_PORT 9090
13
14  void net_set_log(const char *role, int debug_enabled);
15  void net_log(const char *level, const char *format, ...)
16      __attribute__((format(printf, 2, 3)));
17  int net_debug_enabled(void);
18  #if ENABLE_DEBUG
19  #define NET_DEBUG(...) do { if (net_debug_enabled()) net_log("DEBUG", __VA_ARGS__); } while (0)
20  #else
21  #define NET_DEBUG(...) do { } while (0)
22  #endif
23
24  int64_t net_now_ms(void);
25  int net_parse_long(const char *text, long minimum, long maximum, long *value);
26  int net_nonblocking(int fd);
27  int net_connect(const char *host, int port, int timeout_ms);
28  int net_listen(const char *host, int port, int backlog);
29  /* Return 1 for a frame, 0 for clean EOF, -1 with errno on failure. */
30  int net_recv_frame(int fd, char *buffer, size_t capacity, int timeout_ms, int robust);
31  int net_send_frame(int fd, const char *payload, int timeout_ms);
32
33  #endif
```

src/netsys.c

```
 1  #define _POSIX_C_SOURCE 200809L
 2  #include "netsys.h"
 3
 4  #include <arpa/inet.h>
 5  #include <errno.h>
 6  #include <fcntl.h>
 7  #include <limits.h>
 8  #include <netinet/in.h>
 9  #include <poll.h>
10  #include <stdarg.h>
11  #include <stdio.h>
12  #include <stdlib.h>
13  #include <string.h>
14  #include <sys/socket.h>
15  #include <time.h>
16  #include <unistd.h>
17
18  static const char *log_role = "unknown";
19  static int runtime_debug;
20
21  void net_set_log(const char *role, int debug_enabled) {
22      log_role = role;
23      runtime_debug = debug_enabled;
24  }
25
26  int net_debug_enabled(void) { return runtime_debug; }
27
28  void net_log(const char *level, const char *format, ...) {
29      char line[2048];
30      struct timespec now;
31      struct tm stamp;
32      char time_text[32];
```

```
    clock_gettime(CLOCK_REALTIME, &now);
    localtime_r(&now.tv_sec, &stamp);
    strftime(time_text, sizeof(time_text), "%Y-%m-%dT%H:%M:%S", &stamp);
    int used = snprintf(line, sizeof(line), "%s.%03ld %s pid=%ld role=%s ",
                        time_text, now.tv_nsec / 1000000L, level, (long)getpid(), log_role);
    if (used < 0 || (size_t)used >= sizeof(line)) return;
    va_list args;
    va_start(args, format);
    int extra = vsnprintf(line + used, sizeof(line) - (size_t)used, format, args);
    va_end(args);
    if (extra < 0) return;
    size_t size = (size_t)used + (size_t)extra;
    if (size > sizeof(line) - 2) size = sizeof(line) - 2;
    line[size++] = '\n';
    /* One short write keeps individual process log records together. */
    ssize_t ignored = write(STDERR_FILENO, line, size);
    (void)ignored;
}

int64_t net_now_ms(void) {
    struct timespec now;
    if (clock_gettime(CLOCK_MONOTONIC, &now) < 0) return -1;
    return (int64_t)now.tv_sec * 1000 + now.tv_nsec / 1000000;
}

int net_parse_long(const char *text, long minimum, long maximum, long *value) {
    char *end;
    if (!text || !*text) return -1;
    errno = 0;
    long result = strtol(text, &end, 10);
    if (errno || *end || result < minimum || result > maximum) return -1;
    *value = result;
    return 0;
}

int net_nonblocking(int fd) {
    int flags = fcntl(fd, F_GETFL, 0);
    return flags < 0 ? -1 : fcntl(fd, F_SETFL, flags | O_NONBLOCK);
}

static int wait_ready(int fd, short events, int64_t deadline) {
    for (;;) {
        int timeout = -1;
        if (deadline >= 0) {
            int64_t remaining = deadline - net_now_ms();
            if (remaining <= 0) { errno = ETIMEDOUT; return -1; }
            timeout = remaining > INT_MAX ? INT_MAX : (int)remaining;
        }
        struct pollfd item = { .fd = fd, .events = events, .revents = 0 };
        int result = poll(&item, 1, timeout);
        if (result > 0) {
            if (item.revents & POLLNVAL) { errno = EBADF; return -1; }
            /* recv/send reports the precise error for HUP and ERR. */
            return 0;
        }
        if (result == 0) { errno = ETIMEDOUT; return -1; }
        if (errno != EINTR) return -1;
    }
}

static int read_bytes(int fd, void *buffer, size_t length, int64_t deadline,
                      int robust, int allow_clean_eof) {
    size_t received = 0;
    while (received < length) {
        if (wait_ready(fd, POLLIN, deadline) < 0) return -1;
        ssize_t count = recv(fd, (char *)buffer + received, length - received, 0);
        if (count == 0) {
            if (received == 0 && allow_clean_eof) return 0;
            errno = ECONNRESET;
            return -1;
        }
        if (count < 0) {
            if (errno == EINTR || errno == EAGAIN || errno == EWOULDBLOCK) continue;
            return -1;
        }
        received += (size_t)count;
        if (!robust && received < length) { errno = EPROTO; return -1; }
    }
    return 1;
}

int net_recv_frame(int fd, char *buffer, size_t capacity, int timeout_ms, int robust) {
```

```
    uint32_t wire_length;
    int64_t deadline = timeout_ms < 0 ? -1 : net_now_ms() + timeout_ms;
    int result = read_bytes(fd, &wire_length, sizeof(wire_length), deadline, robust, 1);
    if (result <= 0) return result;
    uint32_t length = ntohl(wire_length);
    if (length == 0 || length > NET_MAX_FRAME || (size_t)length >= capacity) {
        errno = EMSGSIZE;
        return -1;
    }
    result = read_bytes(fd, buffer, length, deadline, robust, 0);
    if (result < 0) return -1;
    if (memchr(buffer, '\0', length)) { errno = EPROTO; return -1; }
    buffer[length] = '\0';
    NET_DEBUG("FRAME_RX bytes=%u", length);
    return 1;
}

int net_send_frame(int fd, const char *payload, int timeout_ms) {
    size_t length = strlen(payload);
    if (length == 0 || length > NET_MAX_FRAME) { errno = EMSGSIZE; return -1; }
    unsigned char frame[NET_MAX_FRAME + 4];
    uint32_t wire_length = htonl((uint32_t)length);
    memcpy(frame, &wire_length, 4);
    memcpy(frame + 4, payload, length);
    size_t sent = 0;
    int64_t deadline = timeout_ms < 0 ? -1 : net_now_ms() + timeout_ms;
    while (sent < length + 4) {
        if (wait_ready(fd, POLLOUT, deadline) < 0) return -1;
        ssize_t count = send(fd, frame + sent, length + 4 - sent, MSG_NOSIGNAL);
        if (count < 0) {
            if (errno == EINTR || errno == EAGAIN || errno == EWOULDBLOCK) continue;
            return -1;
        }
        if (count == 0) { errno = EPIPE; return -1; }
        sent += (size_t)count;
    }
    NET_DEBUG("FRAME_TX bytes=%zu", length);
    return 0;
}

static int make_address(const char *host, int port, struct sockaddr_in *address) {
    memset(address, 0, sizeof(*address));
    address->sin_family = AF_INET;
    address->sin_port = htons((uint16_t)port);
    if (inet_pton(AF_INET, host, &address->sin_addr) != 1) { errno = EINVAL; return -1; }
    return 0;
}

int net_connect(const char *host, int port, int timeout_ms) {
    struct sockaddr_in address;
    if (make_address(host, port, &address) < 0) return -1;
    int fd = socket(AF_INET, SOCK_STREAM, 0);
    if (fd < 0) return -1;
    if (net_nonblocking(fd) < 0) goto failure;
    if (connect(fd, (struct sockaddr *)&address, sizeof(address)) == 0) return fd;
    if (errno != EINPROGRESS) goto failure;
    if (wait_ready(fd, POLLOUT, net_now_ms() + timeout_ms) < 0) goto failure;
    int socket_error = 0;
    socklen_t size = sizeof(socket_error);
    if (getsockopt(fd, SOL_SOCKET, SO_ERROR, &socket_error, &size) < 0) goto failure;
    if (socket_error) { errno = socket_error; goto failure; }
    return fd;
failure: {
        int saved = errno;
        close(fd);
        errno = saved;
        return -1;
    }
}

int net_listen(const char *host, int port, int backlog) {
    struct sockaddr_in address;
    if (make_address(host, port, &address) < 0) return -1;
    int fd = socket(AF_INET, SOCK_STREAM, 0);
    if (fd < 0) return -1;
    int enabled = 1;
    if (setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &enabled, sizeof(enabled)) < 0 ||
        net_nonblocking(fd) < 0 ||
        bind(fd, (struct sockaddr *)&address, sizeof(address)) < 0 ||
        listen(fd, backlog) < 0) {
        int saved = errno;
        close(fd);
```

```
197         errno = saved;
198         return -1;
199     }
200     return fd;
201 }
```

src/server.c

```
  1 #define _POSIX_C_SOURCE 200809L
  2 #include "netsys.h"
  3
  4 #include <arpa/inet.h>
  5 #include <errno.h>
  6 #include <getopt.h>
  7 #include <poll.h>
  8 #include <signal.h>
  9 #include <stdio.h>
 10 #include <stdlib.h>
 11 #include <string.h>
 12 #include <sys/socket.h>
 13 #include <sys/sysinfo.h>
 14 #include <sys/types.h>
 15 #include <sys/utsname.h>
 16 #include <sys/wait.h>
 17 #include <time.h>
 18 #include <unistd.h>
 19
 20 #define CHILD_LIMIT 1024
 21 static volatile sig_atomic_t stopping;
 22 static void stop_handler(int number) { (void)number; stopping = 1; }
 23 static void child_handler(int number) { (void)number; }
 24
 25 static int set_handler(int number, void (*handler)(int)) {
 26     struct sigaction action;
 27     memset(&action, 0, sizeof(action));
 28     action.sa_handler = handler;
 29     sigemptyset(&action.sa_mask);
 30     return sigaction(number, &action, NULL);
 31 }
 32
 33 static void usage(FILE *stream, const char *program) {
 34     fprintf(stream, "Usage: %s [--host IPv4] [--port N] [--timeout-ms N] [--max-clients N] [--debug]
    [--unsafe]\n"
 35                     "--unsafe is a teaching mode: single recv, no idle deadline, no child reaping.\n",
    program);
 36 }
 37
 38 static void system_info(char *response, size_t size) {
 39     struct utsname name;
 40     struct sysinfo info;
 41     char hostname[256];
 42     if (uname(&name) < 0 || sysinfo(&info) < 0 || gethostname(hostname, sizeof(hostname)) < 0) {
 43         snprintf(response, size, "ERR SYSINFO %s", strerror(errno));
 44         return;
 45     }
 46     hostname[sizeof(hostname) - 1] = '\0';
 47     snprintf(response, size,
 48              "OK SYSINFO\nhostname=%s\nsysname=%s\nrelease=%s\nmachine=%s\npid=%ld\n"
 49              "uptime_seconds=%ld\nram_total_bytes=%llu\nram_free_bytes=%llu",
 50              hostname, name.sysname, name.release, name.machine, (long)getpid(),
 51              info.uptime, (unsigned long long)info.totalram * info.mem_unit,
 52              (unsigned long long)info.freeram * info.mem_unit);
 53 }
 54
 55 static int handle_command(const char *request, char *response, size_t size) {
 56     if (strcmp(request, "PING") == 0) {
 57         snprintf(response, size, "OK PONG pid=%ld", (long)getpid());
 58     } else if (strcmp(request, "SYSINFO") == 0) {
 59         system_info(response, size);
 60     } else if (strncmp(request, "ECHO ", 5) == 0) {
 61         if (strlen(request + 5) > NET_MAX_FRAME - 3) snprintf(response, size, "ERR ECHO_TOO_LONG");
 62         else snprintf(response, size, "OK %s", request + 5);
 63     } else if (strncmp(request, "SLEEP ", 6) == 0) {
 64         long milliseconds;
 65         if (net_parse_long(request + 6, 0, 10000, &milliseconds) < 0) {
 66             snprintf(response, size, "ERR BAD_ARGUMENT SLEEP accepts 0..10000 milliseconds");
 67         } else {
 68             struct timespec delay = { .tv_sec = milliseconds / 1000, .tv_nsec = (milliseconds % 1000) *
    1000000L };
 69             while (nanosleep(&delay, &delay) < 0 && errno == EINTR) { }
```

```
            snprintf(response, size, "OK SLEPT %ld pid=%ld", milliseconds, (long)getpid());
        }
    } else if (strcmp(request, "QUIT") == 0) {
        snprintf(response, size, "OK BYE");
        return 1;
    } else {
        snprintf(response, size, "ERR UNKNOWN_COMMAND");
    }
    return 0;
}

static void serve_client(int fd, int timeout, int robust, int debug) {
    net_set_log("worker", debug);
    char request[NET_MAX_FRAME + 1], response[NET_MAX_FRAME + 1];
    for (;;) {
        int result = net_recv_frame(fd, request, sizeof(request), robust ? timeout : -1, robust);
        if (result == 0) { NET_DEBUG("PEER_EOF"); break; }
        if (result < 0) {
            int saved = errno;
            net_log("WARN", "FRAME_READ_ERROR errno=%d %s", saved, strerror(saved));
            const char *error = saved == ETIMEDOUT ? "ERR TIMEOUT" :
                                saved == EMSGSIZE ? "ERR BAD_LENGTH" : "ERR BAD_FRAME";
            (void)net_send_frame(fd, error, 300);
            break;
        }
        NET_DEBUG("COMMAND bytes=%zu", strlen(request));
        int quit = handle_command(request, response, sizeof(response));
        if (net_send_frame(fd, response, timeout) < 0) {
            net_log("WARN", "FRAME_WRITE_ERROR errno=%d %s", errno, strerror(errno));
            break;
        }
        if (quit) break;
    }
    close(fd);
    net_log("INFO", "CHILD_DONE");
}

static void remove_child(pid_t *children, size_t *count, pid_t pid) {
    for (size_t index = 0; index < *count; ++index) {
        if (children[index] == pid) {
            children[index] = children[--(*count)];
            return;
        }
    }
}

static void reap_children(pid_t *children, size_t *count) {
    int status;
    pid_t pid;
    while ((pid = waitpid(-1, &status, WNOHANG)) > 0) {
        remove_child(children, count, pid);
        net_log("INFO", "REAP child=%ld active=%zu status=%d", (long)pid, *count, status);
    }
}

int main(int argc, char **argv) {
    const char *host = "127.0.0.1";
    int port = NET_DEFAULT_PORT, timeout = 2000, max_clients = 64, debug = 0, robust = 1;
    static const struct option options[] = {
        {"host", required_argument, NULL, 'H'}, {"port", required_argument, NULL, 'p'},
        {"timeout-ms", required_argument, NULL, 't'}, {"max-clients", required_argument, NULL, 'm'},
        {"debug", no_argument, NULL, 'd'}, {"unsafe", no_argument, NULL, 'u'},
        {"help", no_argument, NULL, 'h'}, {NULL, 0, NULL, 0}
    };
    int option;
    long value;
    while ((option = getopt_long(argc, argv, "H:p:t:m:duh", options, NULL)) != -1) {
        switch (option) {
        case 'H': host = optarg; break;
        case 'p': if (net_parse_long(optarg, 1, 65535, &value) < 0) goto invalid; port = (int)value; break;
        case 't': if (net_parse_long(optarg, 1, 600000, &value) < 0) goto invalid; timeout = (int)value;
break;
        case 'm': if (net_parse_long(optarg, 1, CHILD_LIMIT, &value) < 0) goto invalid; max_clients =
(int)value; break;
        case 'd': debug = 1; break;
        case 'u': robust = 0; break;
        case 'h': usage(stdout, argv[0]); return 0;
        default: goto invalid;
        }
    }
    if (optind != argc) goto invalid;
    net_set_log("server", debug);
```

```
150     if (set_handler(SIGTERM, stop_handler) < 0 || set_handler(SIGINT, stop_handler) < 0 ||
151         set_handler(SIGCHLD, child_handler) < 0 || set_handler(SIGPIPE, SIG_IGN) < 0) {
152         net_log("ERROR", "SIGNAL_SETUP_FAILED %s", strerror(errno)); return 1;
153     }
154     int listener = net_listen(host, port, 128);
155     if (listener < 0) { net_log("ERROR", "LISTEN_FAILED errno=%d %s", errno, strerror(errno)); return 1; }
156     pid_t children[CHILD_LIMIT];
157     size_t count = 0;
158     net_log("INFO", "READY host=%s port=%d robust=%d timeout_ms=%d max_clients=%d compiled_debug=%d",
159             host, port, robust, timeout, max_clients, ENABLE_DEBUG);
160     if (!robust) net_log("WARN", "UNSAFE_TEACHING_MODE partial_read_recovery=off idle_deadline=off
    child_reaping=off");
161     NET_DEBUG("DEBUG_BUILD runtime_debug=on");
162     while (!stopping) {
163         if (robust) reap_children(children, &count);
164         struct pollfd item = { .fd = listener, .events = POLLIN, .revents = 0 };
165         int ready = poll(&item, 1, 100);
166         if (ready < 0) {
167             if (errno == EINTR) continue;
168             net_log("ERROR", "POLL_FAILED %s", strerror(errno));
169             break;
170         }
171         if (ready == 0) continue;
172         struct sockaddr_in peer;
173         socklen_t peer_size = sizeof(peer);
174         int fd = accept(listener, (struct sockaddr *)&peer, &peer_size);
175         if (fd < 0) {
176             if (errno == EINTR || errno == EAGAIN || errno == EWOULDBLOCK || errno == ECONNABORTED)
    continue;
177             net_log("WARN", "ACCEPT_FAILED errno=%d %s", errno, strerror(errno));
178             struct timespec pause = { .tv_sec = 0, .tv_nsec = 100000000L };
179             nanosleep(&pause, NULL);
180             continue;
181         }
182         if (net_nonblocking(fd) < 0) { close(fd); continue; }
183         if (robust) reap_children(children, &count);
184         if (count >= (size_t)max_clients) {
185             net_log("WARN", "CLIENT_LIMIT active=%zu max=%d", count, max_clients);
186             (void)net_send_frame(fd, "ERR SERVER_BUSY", 100);
187             close(fd);
188             continue;
189         }
190         /* A worker must install default termination handlers before an early
191            parent shutdown can send SIGTERM. Otherwise it could inherit the
192            parent's flag-only handler and keep serving while waitpid blocks. */
193         sigset_t termination_signals, old_mask;
194         sigemptyset(&termination_signals);
195         sigaddset(&termination_signals, SIGINT);
196         sigaddset(&termination_signals, SIGTERM);
197         if (sigprocmask(SIG_BLOCK, &termination_signals, &old_mask) < 0) {
198             net_log("ERROR", "SIGNAL_MASK_FAILED %s", strerror(errno));
199             close(fd);
200             continue;
201         }
202         pid_t child = fork();
203         if (child < 0) {
204             int saved = errno;
205             (void)sigprocmask(SIG_SETMASK, &old_mask, NULL);
206             errno = saved;
207             net_log("ERROR", "FORK_FAILED errno=%d %s", errno, strerror(errno));
208             (void)net_send_frame(fd, "ERR SERVER_BUSY", 100);
209             close(fd);
210             continue;
211         }
212         if (child == 0) {
213             close(listener);
214             set_handler(SIGTERM, SIG_DFL);
215             set_handler(SIGINT, SIG_DFL);
216             set_handler(SIGCHLD, SIG_DFL);
217             (void)sigprocmask(SIG_SETMASK, &old_mask, NULL);
218             serve_client(fd, timeout, robust, debug);
219             _exit(0);
220         }
221         close(fd);
222         children[count++] = child;
223         (void)sigprocmask(SIG_SETMASK, &old_mask, NULL);
224         char peer_text[INET_ADDRSTRLEN] = "?";
225         inet_ntop(AF_INET, &peer.sin_addr, peer_text, sizeof(peer_text));
226         net_log("INFO", "FORK child=%ld active=%zu peer=%s:%u", (long)child, count,
227                 peer_text, (unsigned)ntohs(peer.sin_port));
228     }
229     close(listener);
```

```
    /* Shutdown deliberately reaps in both modes so the comparison leaves no debris. */
    for (size_t index = 0; index < count; ++index) (void)kill(children[index], SIGTERM);
    while (count > 0) {
        int status;
        pid_t child = waitpid(-1, &status, 0);
        if (child > 0) remove_child(children, &count, child);
        else if (errno != EINTR) break;
    }
    net_log("INFO", "STOPPED active=%zu", count);
    return 0;
invalid:
    usage(stderr, argv[0]);
    return 2;
}
```

src/client.c

```
#define _POSIX_C_SOURCE 200809L
#include "netsys.h"

#include <errno.h>
#include <getopt.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

static void usage(FILE *stream, const char *program) {
    fprintf(stream, "Usage: %s [--host IPv4] [--port N] [--timeout-ms N] [--debug] COMMAND [ARG ...]\n"
                    "Commands: SYSINFO | PING | ECHO text | SLEEP milliseconds | QUIT\n", program);
}

int main(int argc, char **argv) {
    const char *host = "127.0.0.1";
    int port = NET_DEFAULT_PORT, timeout = 5000, debug = 0;
    static const struct option options[] = {
        {"host", required_argument, NULL, 'H'}, {"port", required_argument, NULL, 'p'},
        {"timeout-ms", required_argument, NULL, 't'}, {"debug", no_argument, NULL, 'd'},
        {"help", no_argument, NULL, 'h'}, {NULL, 0, NULL, 0}
    };
    int option;
    long value;
    while ((option = getopt_long(argc, argv, "+H:p:t:dh", options, NULL)) != -1) {
        switch (option) {
        case 'H': host = optarg; break;
        case 'p': if (net_parse_long(optarg, 1, 65535, &value) < 0) goto invalid; port = (int)value; break;
        case 't': if (net_parse_long(optarg, 1, 600000, &value) < 0) goto invalid; timeout = (int)value;
break;
        case 'd': debug = 1; break;
        case 'h': usage(stdout, argv[0]); return 0;
        default: goto invalid;
        }
    }
    if (optind >= argc) goto invalid;
    net_set_log("client", debug);
    signal(SIGPIPE, SIG_IGN);
    char request[NET_MAX_FRAME + 1] = "";
    size_t used = 0;
    for (int index = optind; index < argc; ++index) {
        size_t length = strlen(argv[index]);
        size_t separator = index == optind ? 0 : 1;
        if (used + separator + length > NET_MAX_FRAME) {
            net_log("ERROR", "REQUEST_TOO_LARGE max=%u", NET_MAX_FRAME);
            return 2;
        }
        if (separator) request[used++] = ' ';
        memcpy(request + used, argv[index], length);
        used += length;
        request[used] = '\0';
    }
    NET_DEBUG("CONNECT host=%s port=%d timeout_ms=%d", host, port, timeout);
    int fd = net_connect(host, port, timeout);
    if (fd < 0) { net_log("ERROR", "CONNECT_FAILED errno=%d %s", errno, strerror(errno)); return 1; }
    if (net_send_frame(fd, request, timeout) < 0) {
        net_log("ERROR", "SEND_FAILED errno=%d %s", errno, strerror(errno)); close(fd); return 1;
    }
    char response[NET_MAX_FRAME + 1];
    int result = net_recv_frame(fd, response, sizeof(response), timeout, 1);
    if (result <= 0) {
```

```
        if (result == 0) net_log("ERROR", "SERVER_CLOSED_WITHOUT_RESPONSE");
        else net_log("ERROR", "RECEIVE_FAILED errno=%d %s", errno, strerror(errno));
        close(fd);
        return 1;
    }
    puts(response);
    close(fd);
    return strncmp(response, "OK", 2) == 0 ? 0 : 1;
invalid:
    usage(stderr, argv[0]);
    return 2;
}
```

Makefile

```
CC ?= gcc
DEBUG ?= 1
ifeq ($(DEBUG),1)
MODE := debug
OPT := -O0 -g3
else ifeq ($(DEBUG),0)
MODE := release
OPT := -O2
else
$(error DEBUG must be 0 or 1)
endif

BUILD := build/$(MODE)
CPPFLAGS += -Iinclude -DENABLE_DEBUG=$(DEBUG)
CFLAGS += -std=c11 -Wall -Wextra -Wpedantic -Werror $(OPT)
LDFLAGS +=

.PHONY: all clean test debug release
all: $(BUILD)/server $(BUILD)/client

$(BUILD):
   mkdir -p $@

$(BUILD)/netsys.o: src/netsys.c include/netsys.h | $(BUILD)
   $(CC) $(CPPFLAGS) $(CFLAGS) -fPIC -c $< -o $@

$(BUILD)/libnetsys.so: $(BUILD)/netsys.o
   $(CC) -shared $(LDFLAGS) -Wl,-soname,libnetsys.so $< -o $@

$(BUILD)/server: src/server.c include/netsys.h $(BUILD)/libnetsys.so
   $(CC) $(CPPFLAGS) $(CFLAGS) $< -L$(BUILD) -lnetsys -Wl,-rpath,'$$ORIGIN' $(LDFLAGS) -o $@

$(BUILD)/client: src/client.c include/netsys.h $(BUILD)/libnetsys.so
   $(CC) $(CPPFLAGS) $(CFLAGS) $< -L$(BUILD) -lnetsys -Wl,-rpath,'$$ORIGIN' $(LDFLAGS) -o $@

debug:
   $(MAKE) DEBUG=1 all

release:
   $(MAKE) DEBUG=0 all

test: all
   python3 tests/run_tests.py

clean:
   rm -rf build
```